MODULE 28 / AI-ZERO-TO-HERO

JavaScript Cheat Sheet

Quick reference from variables to fetch, with production-ready code, visual diagrams and an open-source course behind it.

- Production-ready code
- Visual diagrams
- Tested with Node + jsdom

focusboard.js

```js
import { attempt, fetchJson, createStorage, createLogger } from "./production_kit.js";

const log = createLogger("focusboard");
const storage = createStorage({ namespace: "focusboard", version: 1 });

// never throws: you always get [error, data]
const [err, quote] = await attempt(fetchJson(QUOTE_URL, { timeoutMs: 5000, retries: 2 }));
if (err) log.warn("quote failed, using fallback", err.code);

storage.set("quote", quote ?? FALLBACK_QUOTE, { ttlMs: 60 * 60 * 1000 });
```

GitHub repo | Module 28 | GitHub profile | LinkedIn

repo github.com/Sheharyar-Sarmad/ai-zero-to-hero/
module github.com/Sheharyar-Sarmad/ai-zero-to-hero/tree/main/28_js_basics
profile github.com/Sheharyar-Sarmad/

00 Start here

How to use this cheat sheet

This is the quick-reference companion to **module 28** of the open-source ai-zero-to-hero course. Every section pairs a compact reference with **production-ready code** you can paste into a real project. The windows labelled `production_kit.js` come from one dependency-free ES module that was tested with Node and jsdom.



The module 28 learning path. Each section below maps to one or more of these folders.

terminal SH

```sh
git clone https://github.com/Sheharyar-Sarmad/ai-zero-to-hero.git
cd ai-zero-to-hero/28_js_basics
node 07_functions/06_solutions.js     # run any .js file in the terminal
# open any index.html with the VS Code "Live Server" extension
```

Jump to a section

Folder	Topic	Folder	Topic
01-02	Variables and types	13	Forms and validation
03-04	Operators, coercion, conditionals	14	Timers
06-07	Loops and functions	15	Storage and JSON
08	Arrays	16	Fetch and async
09	Objects	17	Errors and debugging
10	Strings, numbers, money	++	State, gotchas, checklist
11-12	DOM and events		

01-02 Variables and types

let, const, typeof and how values are copied

	var	let	const
Scope	function	block	block
Reassign	yes	yes	no (the binding is fixed)
Before declaration	undefined (hoisted)	ReferenceError (TDZ)	ReferenceError (TDZ)
Use it	avoid	when the value changes	**default choice**

declare.js

```js
const API_URL = "/api/tasks";   // never reassigned
let retries = 0;                // will change
retries += 1;

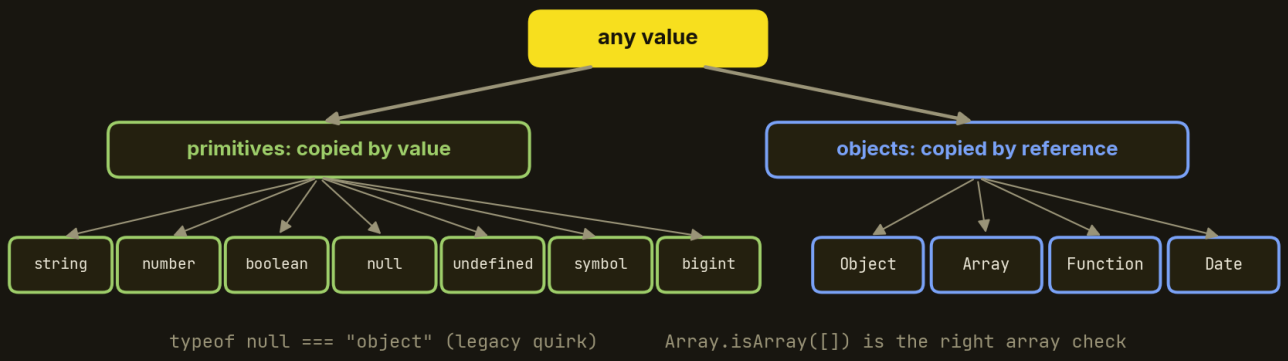
const user = { name: "Ada" };
user.name = "Grace";            // ok: object changed
// user = {};                   // TypeError

const msg = `${user.name}: ${retries} tries`;
```

check.js

```js
typeof 42              // "number"
typeof "hi"            // "string"
typeof undefined       // "undefined"
typeof null            // "object" (quirk)
typeof (() => {})      // "function"
Array.isArray([])      // true
Number.isNaN(NaN)      // true
Object.is(-0, 0)       // false
value == null          // null or undefined
```

Values: primitives vs references



Primitives are copied by value. Objects and arrays are shared by reference.

PRO TIP

`const` protects the **variable**, not the contents. Use `Object.freeze()` for shallow immutability and `structuredClone()` when you need a real copy.

03-04 Operators, coercion, conditionals

compare strictly, convert explicitly, return early

operators.js

```js
const port = config.port ?? 3000;       // fallback only for null/undefined
const name = input || "Guest";          // fallback for any falsy value
const city = user?.address?.city;       // safe chain, no TypeError
user.onLogin?.();                       // call only if it exists
ready && start();                       // short-circuit
const label = score >= 50 ? "Pass" : "Retry";
2 ** 10; 7 % 3; Math.trunc(-4.7);       // 1024  1  -4
```

Falsy values (exactly eight)

false	0	-0	0n	""	null	undefined	NaN

Everything else is truthy, including `"0"`, `"false"`, `[]`, `{}` and functions.

Expression	Result	Why
`"5" + 1`	`"51"`	`+` prefers strings
`"5" - 1`	`4`	`-` converts to numbers
`true + 1`	`2`	booleans become 1 or 0
`[] + []`	`""`	arrays become strings
`null + 1`	`1`	`null` becomes 0
`undefined + 1`	`NaN`	`undefined` becomes NaN
`0 == ""`	`true`	loose equality coerces. Use `===`
`NaN === NaN`	`false`	use `Number.isNaN()`

guard-clauses.js

```js
function getDiscount(user, total) {
  if (!user) return 0;                              // guard clauses first
  if (!Number.isFinite(total) || total <= 0) return 0;
  if (user.plan === "pro") return total * 0.2;
  return total > 100 ? total * 0.1 : 0;             // happy path last
}
```

06-07 Loops and functions

pick the loop that states your intent

I need to...	Use
use the index	`for (let i = 0; i < n; i++)`
go through every value	`for (const item of items)`
go through an object	`for (const [k, v] of Object.entries(obj))`
repeat until something changes	`while (condition)`
transform / filter / total up	`map` / `filter` / `reduce`
stop early	`for...of` + `break`, or `find` / `some`

functions.js JS

```js
function add(a, b = 0) {
  return a + b;              // hoisted
}
const mul = (a, b) => a * b; // no own `this`
const sum = (...nums) =>
  nums.reduce((t, n) => t + n, 0);

const makeCounter = () => {
  let n = 0;                 // private (closure)
  return () => ++n;
};
const next = makeCounter();
next(); next();              // 1, 2
```

patterns.js JS

```js
// callbacks and higher-order functions
const byPriority = (a, b) => a.p - b.p;
tasks.sort(byPriority);

setTimeout(() => save(draft), 500);

button.addEventListener("click", onSave);

// pure: same input, same output
const total = (items) =>
  items.reduce((t, i) => t + i.cents, 0);

// impure: touches the outside world
let count = 0;
const bump = () => ++count;
```

production_kit.js / functions JS

```js
export const pipe = (...fns) => (input) => fns.reduce((value, fn) => fn(value), input);

export function memoize(fn, keyFn = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  const memoized = (...args) => {
    const key = keyFn(...args);
    if (!cache.has(key)) cache.set(key, fn(...args));
    return cache.get(key);
  };
  memoized.clear = () => cache.clear();
  return memoized;
}

export const once = (fn) => {
  let called = false, result;
  return (...args) => (called ? result : ((called = true), (result = fn(...args))));
};

export function debounce(fn, delay = 300) {
  let timer, lastArgs;
  const debounced = (...args) => {
    lastArgs = args;
    clearTimeout(timer);
    timer = setTimeout(() => { timer = undefined; fn(...lastArgs); }, delay);
  };
  debounced.cancel = () => { clearTimeout(timer); timer = undefined; };
  debounced.flush = () => {
    if (timer === undefined) return;
    clearTimeout(timer); timer = undefined; fn(...lastArgs);
  };
  return debounced;
}

export function throttle(fn, interval = 200) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= interval) { last = now; fn(...args); }
  };
}
```

PRO TIP
Debounce waits for a **pause** (search box). Throttle caps the **rate** (scroll, resize). `debounce.cancel()` and `.flush()` make cleanup and tests easy.

08 Arrays

transform data without mutating it

Mutates the original	Returns something new
`push` `pop` `shift` `unshift` `splice` `sort` `reverse` `fill`	`slice` `map` `filter` `reduce` `concat` `flat` `join` `toSorted` `toReversed`

Method	Does	Returns
`map(fn)`	transform each item	new array, same length
`filter(fn)`	keep items that pass	new, shorter array
`reduce(fn, init)`	fold into one value	any value
`find(fn)` / `findIndex`	first match	item / index (-1)
`some` / `every`	any or all pass	boolean
`includes(x)`	contains a value	boolean
`flat()` / `flatMap(fn)`	unnest / map then unnest	new array

immutable-arrays.js — JS

```js
const next = [...items, newItem];                          // add
const without = items.filter((it) => it.id !== id);        // remove
const patched = items.map((it) => (it.id === id ? { ...it, done: true } : it));  // update
const sorted = [...items].sort((a, b) => a.n - b.n);       // sort a copy
const [first, ...rest] = items;                            // destructure
const last = items.at(-1);                                 // last item
[10, 9, 1].sort();                                         // [1, 10, 9] (strings!)
```

production_kit.js / collections — JS

```js
export const groupBy = (items, keyFn) =>
  items.reduce((acc, item) => {
    (acc[keyFn(item)] ??= []).push(item);
    return acc;
  }, {});

export const unique = (items, keyFn = (x) => x) => {
  const seen = new Set();
  return items.filter((item) => {
    const key = keyFn(item);
    return seen.has(key) ? false : (seen.add(key), true);
  });
};

export const chunk = (items, size) =>
  Array.from({ length: Math.ceil(items.length / size) }, (_, i) => items.slice(i * size, i * size + size));

export const sortBy = (items, keyFn, dir = "asc") =>
  [...items].sort((a, b) => {
    const x = keyFn(a), y = keyFn(b);
    const order = x < y ? -1 : x > y ? 1 : 0;
    return dir === "asc" ? order : -order;
  });

// Immutable updates: return new arrays and objects, never mutate shared state.
```

```
export const updateById = (items, id, patch) =>
  items.map((it) => (it.id === id ? { ...it, ...patch } : it));
export const removeById = (items, id) => items.filter((it) => it.id !== id);
export const pick = (obj, keys) => Object.fromEntries(keys.filter((k) => k in obj).map((k) => [k, obj[k]]));
export const omit = (obj, keys) =>
  Object.fromEntries(Object.entries(obj).filter(([k]) => !keys.includes(k)));
```

09 Objects

copy, merge and read safely

objects.js — JS

```
const { title, priority = "medium", ...others } = task;   // destructure with default
const merged = { ...defaults, ...overrides };              // shallow merge
const deep = structuredClone(state);                       // deep copy
const key = "done";  const patch = { [key]: true };       // computed key
Object.entries(obj).map(([k, v]) => `${k}=${v}`);          // object -> array
Object.freeze(CONFIG);                                     // shallow immutable
const name = task.owner?.name ?? "unassigned";             // safe nested read

const timer = { seconds: 60, tick() { this.seconds -= 1; } };   // method + this
setTimeout(timer.tick, 1000);          // BUG: loses `this`. Use () => timer.tick()
```

PRO TIP

Spread and `Object.assign` copy **one level**. Nested objects are still shared, so use `structuredClone()` for deep copies. `JSON.parse(JSON.stringify(x))` drops `undefined`, functions and Dates.

10 Strings, numbers, money

text tools and floating-point safety

strings-numbers.js — JS

```
"  Hi  ".trim().toUpperCase();            // "HI"
"a-b-c".split("-").join("+");             // "a+b+c"
"7".padStart(2, "0");                     // "07"
text.replaceAll(" ", "-");  text.at(-1);  text.slice(0, 10);
(1234.5).toFixed(2);                      // "1234.50"  (a string!)
Number("12px");  parseInt("12px", 10);    // NaN  12
Math.floor(Math.random() * 6) + 1;        // 1..6
0.1 + 0.2 === 0.3;                        // false: floating point
new Date(2026, 0, 31);                    // months are 0-based: this is January
```

production_kit.js / numbers and money — JS

```
export const toNumber = (value, fallback = 0) => {
  const n = typeof value === "string" && value.trim() === "" ? NaN : Number(value);
  return Number.isFinite(n) ? n : fallback; // blocks NaN, Infinity and empty strings
};

export const clamp = (n, min, max) => Math.min(Math.max(n, min), max);

// Money: store integer cents, format only at the edge (0.1 + 0.2 !== 0.3).
export const toCents = (amount) => Math.round(toNumber(amount) * 100);
export const formatMoney = (cents, currency = "USD", locale = "en-US") =>
  new Intl.NumberFormat(locale, { style: "currency", currency }).format(cents / 100);

export function formatTime(totalSeconds) {
```

```
  const safe = Math.max(0, Math.floor(toNumber(totalSeconds)));
  return `${String(Math.floor(safe / 60)).padStart(2, "0")}:${String(safe % 60).padStart(2, "0")}`;
}
```

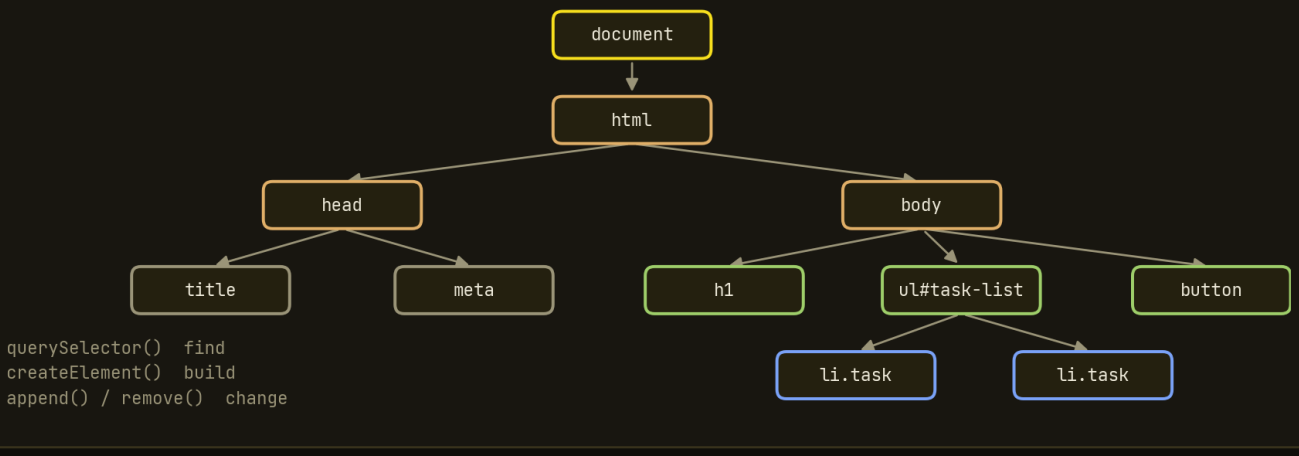
PRO TIP
Never store money as a float. Keep **integer cents**, do the maths on integers, and format with `Intl.NumberFormat` only when you display it.

11-12 DOM and events

build safely, listen once, delegate

The DOM is a tree

find a node, change it, the browser repaints



querySelector finds a node, createElement builds one, append and remove change the tree.

Goal	Use
select	`querySelector` / `querySelectorAll` / `getElementById`
text	`textContent` (safe). `innerHTML` only for trusted HTML
classes and data	`classList.add / remove / toggle`, `dataset.key`
create and insert	`createElement`, `append`, `prepend`, `before`, `remove`
listen	`addEventListener(type, handler, { once: true })`
control flow	`event.preventDefault()`, `event.stopPropagation()`

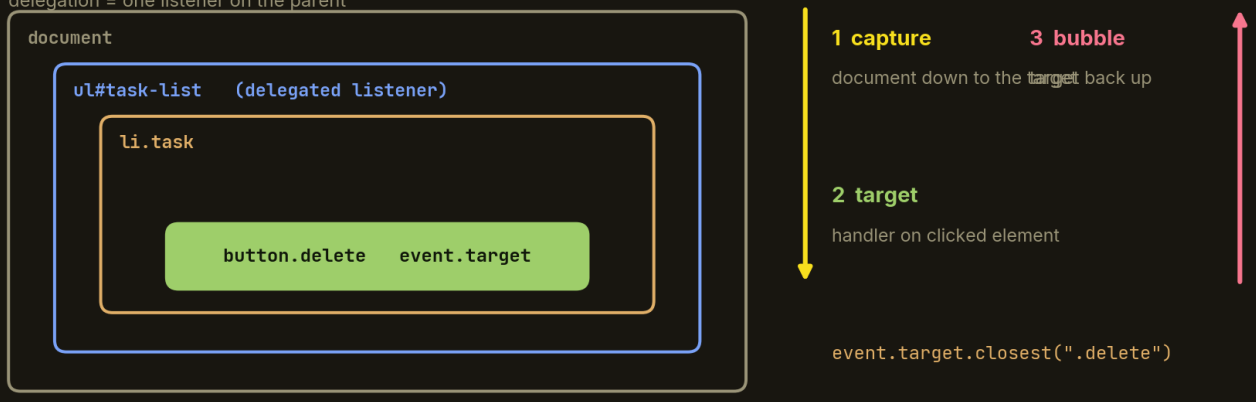
dom-basics.js JS

```
const list = document.querySelector("#task-list");
const li = document.createElement("li");
li.textContent = userInput;                 // safe even if it contains <script>
li.classList.toggle("done", task.done);
li.dataset.id = String(task.id);
list.append(li);

list.addEventListener("click", (e) => {
  const btn = e.target.closest(".delete");   // delegation: one listener, many children
  if (btn) remove(Number(btn.closest("li").dataset.id));
});
```

Events: capture, target, bubble

delegation = one listener on the parent



Events capture down, run on the target, then bubble up. Delegation uses the bubbling phase.

production_kit.js / dom helpers JS

```js
export const $ = (selector, root = document) => root.querySelector(selector);
export const $$ = (selector, root = document) => [...root.querySelectorAll(selector)];

const HTML_ESCAPES = { "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;", "'": "&#39;" };
export const escapeHtml = (value) => String(value).replace(/[&<>"']/g, (ch) => HTML_ESCAPES[ch]);

/** Build elements safely: text goes through text nodes, never innerHTML. */
export function h(tag, props = {}, ...children) {
  const el = document.createElement(tag);
  for (const [key, value] of Object.entries(props)) {
    if (key === "class") el.className = value;
    else if (key === "dataset") Object.assign(el.dataset, value);
    else if (key.startsWith("on") && typeof value === "function") {
      el.addEventListener(key.slice(2).toLowerCase(), value); // onClick -> "click"
    }
    else if (value !== false && value != null) el.setAttribute(key, value);
  }
  el.append(...children.flat().map((c) => (c instanceof Node ? c : document.createTextNode(String(c)))));
  return el;
}

/** One listener on the parent handles every matching child, including future ones. */
export function delegate(parent, type, selector, handler) {
  const listener = (event) => {
    const target = event.target.closest(selector);
    if (target && parent.contains(target)) handler(event, target);
  };
  parent.addEventListener(type, listener);
  return () => parent.removeEventListener(type, listener);
}

/** Screen-reader announcement through an aria-live region. */
export function announce(message, region = document.getElementById("live-region")) {
  if (region) region.textContent = message;
}
```

13 Forms and validation

accessible, testable, never trusted

index.html HTML

```html
<form id="task-form" novalidate>
  <label for="title">Task</label>
  <input id="title" name="title"
         required minlength="3"
         aria-describedby="title-error" />
  <small id="title-error"
         role="alert"></small>
  <button type="submit">Add</button>
</form>
<p id="live-region" aria-live="polite"
   class="sr-only"></p>
```

form.js JS

```js
const rules = {
  title: [required(), minLength(3), maxLength(80)],
  due:   [notInPast()],
};

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const values = Object.fromEntries(
    new FormData(form));
  const { valid, errors } = validate(values, rules);
  titleError.textContent = errors.title ?? "";
  if (!valid) return titleInput.focus();
  addTask(values);
  announce("Task added");
  form.reset();
});
```

production_kit.js / validation rules JS

```js
export function validate(values, rules) {
  const errors = {};
  for (const [field, checks] of Object.entries(rules)) {
    for (const check of checks) {
      const message = check(values[field], values);
      if (message) { errors[field] = message; break; } // first failing rule wins
    }
  }
  return { valid: Object.keys(errors).length === 0, errors };
}

const text = (v) => String(v ?? "").trim();
export const required = (msg = "This field is required.") => (v) => (text(v) ? null : msg);
export const minLength = (n, msg = `Enter at least ${n} characters.`) => (v) =>
  text(v).length >= n ? null : msg;
export const maxLength = (n, msg = `Keep it under ${n} characters.`) => (v) =>
  text(v).length <= n ? null : msg;
export const matches = (re, msg = "Invalid format.") => (v) => (re.test(text(v)) ? null : msg);
export const todayISO = () => {
  const d = new Date();                                    // local date, not UTC
  return new Date(d.getTime() - d.getTimezoneOffset() * 60000).toISOString().slice(0, 10);
};
export const notInPast = (msg = "Pick today or a future date.") => (v) =>
  !v || v >= todayISO() ? null : msg;
```

PRO TIP

Client-side validation is for **usability**. The server must validate again. Say how to fix the problem ("Enter at least 3 characters"), not just that it is wrong.

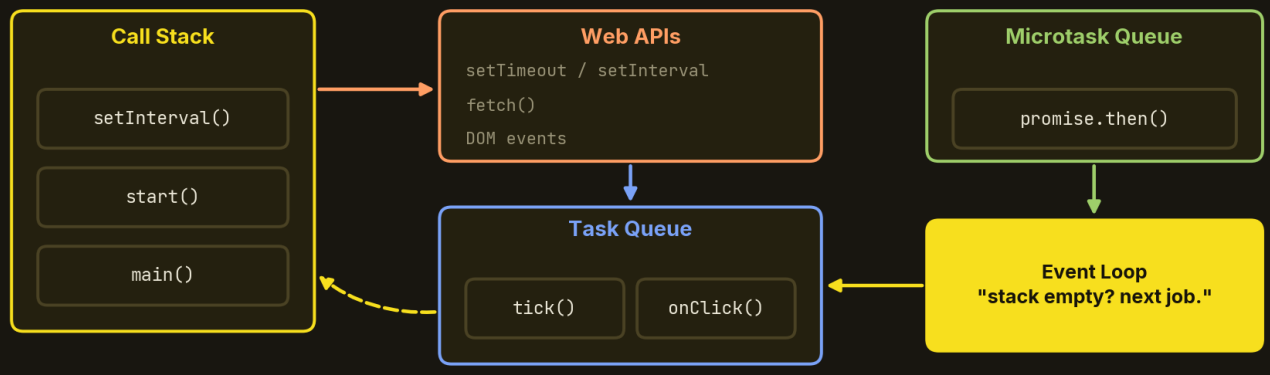
14 Timers

schedule work without leaking or drifting

API	Runs	Remember
`setTimeout(fn, ms)`	once	store the id and call `clearTimeout`
`setInterval(fn, ms)`	repeatedly	call `clearInterval`. Guard against double start
recursive `setTimeout`	repeatedly, no overlap	next run waits for the previous one to finish
`requestAnimationFrame`	once per frame	use for animation, never `setInterval`
`debounce` / `throttle`	wrapped function	see the functions section

The event loop

timers, events and promises all queue behind the call stack



Microtasks (promises) run before the next task. Long synchronous code blocks everything.

Timers, events and promises wait in queues. The event loop moves them onto the call stack when it is empty.

production_kit.js / countdown without drift — JS

```js
export class Countdown {
  #id = null;
  #endAt = 0;
  #left;

  constructor(seconds, { onTick = () => {}, onDone = () => {} } = {}) {
    this.total = seconds;
    this.#left = seconds;
    this.onTick = onTick;
    this.onDone = onDone;
  }

  get running() { return this.#id !== null; }
  get remaining() { return this.#left; }

  start() {
    if (this.running) return;                          // never two intervals
    this.#endAt = Date.now() + this.#left * 1000;      // real clock: no drift
    this.#id = setInterval(() => this.#tick(), 250);
  }

  #tick() {
    this.#left = Math.max(0, Math.ceil((this.#endAt - Date.now()) / 1000));
    this.onTick(this.#left);
    if (this.#left === 0) { this.pause(); this.#left = this.total; this.onDone(); }
  }
```

```
27
28    pause() {
29      if (!this.running) return;
30      this.#left = Math.max(0, Math.ceil((this.#endAt - Date.now()) / 1000));
31      clearInterval(this.#id);
32      this.#id = null;
33    }
34
35    reset(seconds = this.total) {
36      this.pause();
37      this.total = seconds;
38      this.#left = seconds;
39      this.onTick(this.#left);
40    }
41  }
```

PRO TIP
A timer delay is a **minimum**, not a promise. Count with `Date.now()` (as `Countdown` does) and the clock stays right even when the tab is busy or throttled.

15 Storage and JSON

persist data that survives refresh, bad data and upgrades

	localStorage	sessionStorage	Cookies	IndexedDB
Lifetime	until cleared	tab closes	set by expiry	until cleared
Size	about 5 MB	about 5 MB	about 4 KB	large
Sent to server	no	no	yes, every request	no
API	sync, strings only	sync, strings only	string	async, structured
Best for	preferences, drafts	one-tab flows	server sessions	big offline data

production_kit.js / versioned storage with TTL — JS

```
1   export function createStorage(options) {
2     const { namespace, version = 1, migrate = (data) => data, backend = globalThis.localStorage } = options;
3     const fullKey = (key) => `${namespace}.${key}`;
4     return {
5       get(key, fallback = null) {
6         try {
7           const raw = backend.getItem(fullKey(key));
8           if (raw === null) return fallback;
9           const { v, exp, data } = JSON.parse(raw);
10          if (exp && Date.now() > exp) { backend.removeItem(fullKey(key)); return fallback; } // expired
11          return v === version ? data : migrate(data, v);                                    // old schema
12        } catch (error) {
13          console.error(`storage.get("${key}") failed`, error);
14          return fallback; // corrupted JSON or storage blocked
15        }
16      },
17      set(key, data, { ttlMs } = {}) {
18        try {
19          const envelope = { v: version, exp: ttlMs ? Date.now() + ttlMs : null, data };
20          backend.setItem(fullKey(key), JSON.stringify(envelope));
21          return true;
22        } catch (error) {
23          console.error(`storage.set("${key}") failed`, error); // quota exceeded, private mode
24          return false;
25        }
26      },
```

```
    remove(key) {
      try { backend.removeItem(fullKey(key)); } catch (error) { console.error(error); }
    },
  };
}
```

storage-usage.js JS

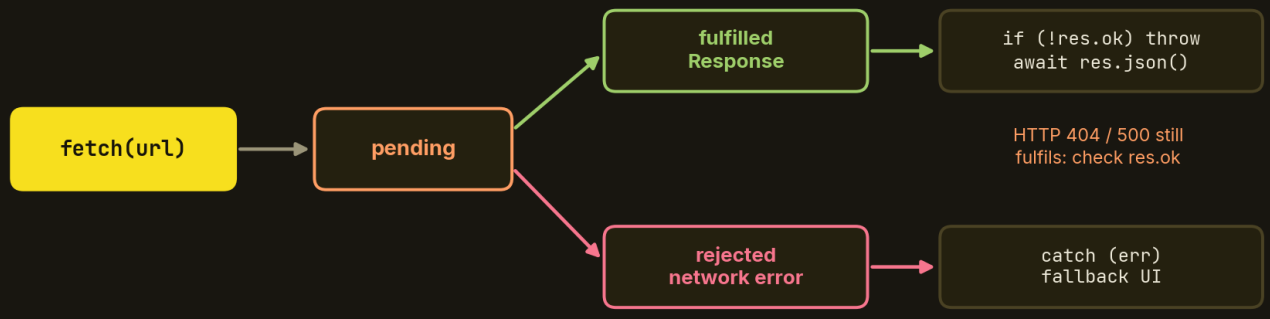
```
const storage = createStorage({
  namespace: "focusboard", version: 2,
  migrate: (data, from) => (from === 1 ? { ...data, tags: [] } : data),   // upgrade old data
});
storage.set("tasks", tasks);
storage.set("quote", quote, { ttlMs: 60 * 60 * 1000 });    // expires in one hour
const saved = storage.get("tasks", []);                    // fallback if missing or corrupted
```

SECURITY
Never store passwords, tokens or personal data in web storage. Any script on the page can read it.

16 Fetch and async

timeouts, retries, cancellation, concurrency

fetch() and promise states



fetch only rejects on network failure. A 404 or 500 still fulfils, so always check response.ok.

production_kit.js / fetch with timeout, retry and cancel JS

```
export const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

export async function fetchJson(url, opts = {}) {
  const { timeoutMs = 8000, retries = 2, baseDelayMs = 400, signal, parse = (r) => r.json(), ...init } =
    opts;
  let lastError;

  for (let attempt = 0; attempt <= retries; attempt++) {
    const controller = new AbortController();
    const forwardAbort = () => controller.abort();
    signal?.addEventListener("abort", forwardAbort, { once: true });
    let timedOut = false;
    const timer = setTimeout(() => { timedOut = true; controller.abort(); }, timeoutMs);

    try {
      if (signal?.aborted) throw new AppError("Request cancelled", { code: "ABORTED" });
      const response = await fetch(url, { ...init, signal: controller.signal });
```

```
      if (!response.ok) {
        throw new AppError(`HTTP ${response.status}`, { code: "HTTP_ERROR", status: response.status });
      }
      return await parse(response);
    } catch (err) {
      lastError = timedOut ? new AppError("Request timed out", { code: "TIMEOUT", cause: err })
        : err.name === "AbortError" ? new AppError("Request cancelled", { code: "ABORTED", cause: err })
        : err;
      const { status, code } = lastError;
      const clientError = status >= 400 && status < 500 && ![408, 429].includes(status);
      if (code === "ABORTED" || clientError || attempt === retries) break; // 404 will not fix itself
      await sleep(baseDelayMs * 2 ** attempt * (0.5 + Math.random() / 2));           // backoff + jitter
    } finally {
      clearTimeout(timer);
      signal?.removeEventListener("abort", forwardAbort);
    }
  }
  throw lastError;
}

export function withTimeout(promise, ms, message = "Timed out") {
  let timer;
  const timeout = new Promise((_, reject) => {
    timer = setTimeout(() => reject(new AppError(message, { code: "TIMEOUT" })), ms);
  });
  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
}

/** Run at most `concurrency` async tasks at once (be kind to APIs). */
export function pLimit(concurrency = 3) {
  const queue = [];
  let active = 0;
  const next = () => {
    if (active >= concurrency || queue.length === 0) return;
    const { task, resolve, reject } = queue.shift();
    active++;
    task().then(resolve, reject).finally(() => { active--; next(); });
  };
  return (task) => new Promise((resolve, reject) => { queue.push({ task, resolve, reject }); next(); });
}

export async function settleAll(promises) {
  const results = await Promise.allSettled(promises);
  return {
    ok: results.filter((r) => r.status === "fulfilled").map((r) => r.value),
    failed: results.filter((r) => r.status === "rejected").map((r) => r.reason),
  };
}
```

async-patterns.js JS

```
// slow: each await waits for the previous one
const a = await getUser();  const b = await getPosts();
// fast: start both, wait for both
const [user, posts] = await Promise.all([getUser(), getPosts()]);
// tolerant: keep what worked
const { ok, failed } = await settleAll(ids.map(getItem));
// polite: at most 3 requests in flight
const limit = pLimit(3);
const pages = await Promise.all(urls.map((u) => limit(() => fetchJson(u))));
// BUG: forEach ignores await. Use for...of or Promise.all
```

search-as-you-type.js JS

```js
let controller;
async function search(q) {
  controller?.abort();                          // cancel the previous request
  controller = new AbortController();
  const [err, data] = await attempt(fetchJson(buildUrl(q), { signal: controller.signal }));
  if (err?.code === "ABORTED") return;          // superseded by a newer search: ignore
  render(err ? [] : data);
}
const onType = debounce((e) => search(e.target.value), 250);
```

Status	Meaning	Retry?
200 / 201 / 204	ok / created / no content	n/a
400 / 422	bad request / invalid data	no. Fix the request
401 / 403	not signed in / not allowed	no. Re-authenticate
404 / 409	not found / conflict	no
408 / 429	timeout / rate limited	**yes**, with backoff (honour `Retry-After`)
500 / 502 / 503 / 504	server trouble	**yes**, with backoff and jitter

17 Errors and debugging

expect failure, report it clearly, recover

production_kit.js / typed errors and attempt() JS

```js
export class AppError extends Error {
  constructor(message, { code = "APP_ERROR", status, cause } = {}) {
    super(message, { cause });
    this.name = "AppError";
    this.code = code;
    this.status = status;
  }
}

export function invariant(condition, message, code = "INVARIANT") {
  if (!condition) throw new AppError(message, { code });
}

/** Never throws. Always returns [error, data]. Removes try/catch noise at call sites. */
export async function attempt(taskOrPromise) {
  try {
    const data = await (typeof taskOrPromise === "function" ? taskOrPromise() : taskOrPromise);
    return [null, data];
  } catch (error) {
    return [error, null];
  }
}
```

error-handling.js JS

```js
class ValidationError extends AppError {
  constructor(field, message) { super(message, { code: "VALIDATION" }); this.field = field; }
}

try {
  await saveTask(task);
```

```
} catch (error) {
  if (error instanceof ValidationError) showFieldError(error.field, error.message);
  else if (error.code === "TIMEOUT") toast("The server is slow. Try again.");
  else throw error;                       // unknown: never swallow it
} finally {
  setLoading(false);                      // always runs
}
```

production_kit.js / logger and global handlers JS

```
const LEVELS = { debug: 10, info: 20, warn: 30, error: 40, silent: 99 };

export function createLogger(scope, level = "info") {
  const enabled = (name) => LEVELS[name] >= LEVELS[level];
  const log = (name) => (...args) => {
    if (enabled(name)) console[name === "debug" ? "log" : name](`[${scope}]`, ...args);
  };
  return { debug: log("debug"), info: log("info"), warn: log("warn"), error: log("error") };
}

/** Last line of defence for anything nobody caught. */
export function registerGlobalErrorHandlers(report = console.error) {
  window.addEventListener("error", (e) => report("Uncaught error:", e.error ?? e.message));
  window.addEventListener("unhandledrejection", (e) => report("Unhandled rejection:", e.reason));
}
```

Console	Use it for
`console.log / warn / error`	levels: info, warning, failure (with stack)
`console.table(rows)`	arrays of objects as a grid
`console.group()` / `groupEnd()`	collapse related logs
`console.time(label)` / `timeEnd(label)`	measure how long something takes
`console.assert(cond, msg)`	log only when the condition is false
`console.trace()`	print the call stack here

DevTools action	Windows / Linux	Mac
open DevTools	`F12`	`Cmd+Opt+I`
inspect an element	`Ctrl+Shift+C`	`Cmd+Shift+C`
open a file in Sources	`Ctrl+P`	`Cmd+P`
resume / step over / step into / step out	`F8` / `F10` / `F11` / `Shift+F11`	same keys
command menu	`Ctrl+Shift+P`	`Cmd+Shift+P`

The debugging loop

1 Reproduce	2 Isolate	3 Hypothesise	4 Test it	5 Fix and guard
make it fail every time	shrink to the smallest case	read the message and stack first	breakpoint, watch, log one thing	fix, then add a check so it never returns

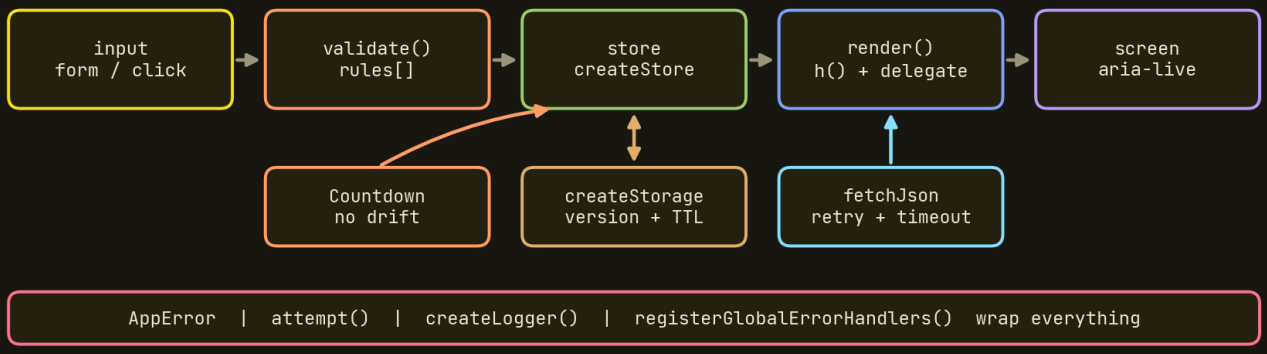
++

State, gotchas, checklist

keep state predictable and ship with confidence

Production data flow

every arrow is a function in production_kit.js



FocusBoard data flow. Every arrow is a function from production_kit.js.

production_kit.js / emitter and store — JS

```js
export function createEmitter() {
  const handlers = new Map();
  return {
    on(event, fn) {
      if (!handlers.has(event)) handlers.set(event, new Set());
      handlers.get(event).add(fn);
      return () => handlers.get(event)?.delete(fn); // unsubscribe
    },
    emit(event, payload) { handlers.get(event)?.forEach((fn) => fn(payload)); },
  };
}

/** Tiny predictable store: one state object, changed only through a reducer. */
export function createStore(initialState, reducer) {
  let state = initialState;
  const listeners = new Set();
  return {
    getState: () => state,
    dispatch(action) {
      const next = reducer(state, action);
      if (next === state) return;                // no change, no re-render
      state = next;
      listeners.forEach((listener) => listener(state, action));
    },
    subscribe(listener) { listeners.add(listener); return () => listeners.delete(listener); },
  };
}
```

store-usage.js — JS

```js
const store = createStore({ tasks: [], filter: "all" }, (state, action) => {
  switch (action.type) {
    case "add":
      return { ...state, tasks: [action.task, ...state.tasks] };
    case "toggle":
      return { ...state, tasks: updateById(state.tasks, action.id, { done: action.done }) };
```

```
      case "filter":
        return { ...state, filter: action.value };
      default:
        return state;                    // same object: subscribers are not called
    }
  });
  store.subscribe((state) => { render(state); storage.set("tasks", state.tasks); });
  store.dispatch({ type: "add", task: { id: 1, title: "Ship it", done: false } });
```

Top gotchas

Gotcha	Fix
`0.1 + 0.2 !== 0.3`	integer cents, or compare with a tolerance
`[10, 9, 1].sort()` is alphabetical	`sort((a, b) => a - b)`
`parseInt("08")` without radix	`parseInt(text, 10)` or `Number(text)`
`forEach` does not wait for `await`	`for...of` or `Promise.all(items.map(...))`
arrow function as object method	use method shorthand when you need `this`
`var` in a loop captured by callbacks	use `let` or `const`
mutating state other code reads	copy first: `[...a]`, `{ ...o }`, `structuredClone`
`JSON.stringify` drops `undefined` and functions	design plain-data objects
`fetch` resolves on HTTP 404	check `response.ok`
forgetting `await`	you hold a Promise, not the data

Production checklist

Area	Do this
Security	`textContent` not `innerHTML`, validate on the server, no secrets in storage, HTTPS, Content-Security-Policy
Reliability	timeouts, bounded retries with backoff, fallbacks, typed errors, global error handlers
Performance	debounce and throttle, batch DOM updates, clear timers and listeners, avoid huge synchronous loops
Accessibility	labels on every input, focus management, `aria-live` for updates, visible focus, enough contrast
Maintainability	small pure functions, named constants, JSDoc, Prettier + ESLint, tests for edge cases (empty, huge, invalid)
Process	commit small, document with a README, review error paths first

^^

Links

the course, the module, and the author

Resource	Link
Course repository (ai-zero-to-hero)	github.com/Sheharyar-Sarmad/ai-zero-to-hero
Module 28: JavaScript Basics	github.com/.../tree/main/28_js_basics
GitHub profile	github.com/Sheharyar-Sarmad
LinkedIn	linkedin.com/in/sheharyar-sarmad-9b7736289

Module 28 folders

Open on GitHub	Open on GitHub
01 foundation	11 DOM manipulation
02 variables and data types	12 event listeners
03 operators	13 forms validation
04 type conversion	14 timers and intervals
06 loops	15 localStorage and JSON
07 functions	16 fetch API
08 arrays	17 error handling and debugging
09 objects	capstone project
10 strings, numbers, math	

Learn more

Resource	Use it for	Link
MDN Web Docs: JavaScript	reference for every method here	developer.mozilla.org
javascript.info	long-form modern JavaScript	javascript.info
Chrome DevTools docs	breakpoints, network, performance	developer.chrome.com/docs/devtools
Can I use	browser support checks	caniuse.com
web.dev	performance and accessibility guides	web.dev

Found a mistake or want to add something? Open an issue or pull request on the repository. The course is open source.